

Deep learning-based software bug classification

Jyoti Prakash Meher^{*}, Sourav Biswas, Rajib Mall

Department of Computer Science and Engineering, Indian Institute of Technology, Kharagpur, 721302, West Bengal, India

ARTICLE INFO

Keywords:

Automatic classification
Bug analysis
Self attention
Transfer learning

ABSTRACT

Context: Accurate classification of bugs can help accelerate the bug triage process, code inspection, and repair activities. In this context, many machine learning techniques have been proposed to classify bugs. The expressive power of deep learning could be used to further improve classification.

Objective: We propose a novel deep learning-based bug classification approach.

Methods: We first build a bug taxonomy with eight bug classes, each characterized by a set of keywords. Subsequently, we heuristically annotate a moderately large set (~1.36M) of software bug resolution reports using an earth-mover distance technique based on the keywords. Finally, we use four attention-based classification techniques to classify these curated bugs.

Results: Our experiments on a carefully collected dataset indicate that our proposed technique achieved a mean F1-Score of 84.78% and a mean macro-average ROC of 98.25%.

Conclusion: Our proposed approach was observed to outperform the existing techniques by 16.88% on an average in terms of F1-Score for the considered dataset.

1. Introduction

Software development is a complex and challenging process that has traditionally relied heavily on manual activities such as writing program source code and maintaining it. These manual activities are susceptible to errors [1]. Even careful and experienced programmers commit mistakes [2]. Software bugs, therefore have remained unavoidable during development and maintenance of commercial software. The effects of bugs can range from minor irritations to devastating situations such as the Equifax data breach, the WannaCry ransomware attack [3], and the Blue Screen of Death [4]. In complex software, a large number of bug reports are raised during software development as well as the maintenance phase. To effectively handle these bugs, it is necessary to first classify them. Not only does manual classification of bug reports require sufficient knowledge and expertise in the field, but it also consumes significant amount of time and effort. Moreover, in a typical deadline-bounded delivery process of software products, it is imperative to automate the bug classification task.

It is widely acknowledged that accurate classification of bugs can help make the bug triage process, code inspection, and repair activities more efficient and effective [9]. In this context, it is worth mentioning that a bug resolution report is a resource-rich artifact that contains different types of information about the bug. An example of a real bug report is shown in Fig. 1. A crucial part of a bug resolution report is the summary that captures important information about the

bug [5], which is usually written in natural language. On the basis of the information contained in a bug resolution report, numerous bug classification techniques have been proposed in the literature [6–11].

Over the last few years, deep learning techniques are gaining attention from researchers and have been used to classify bugs using bug resolution reports [12]. In the reported work, bug classification is carried out using multi-label text classification techniques [13] used in the area of natural language processing (NLP). Toward this end, researchers have attempted to use several machine learning techniques [6,10]. However, there appears to be sufficient scope for further performance improvement by deploying deep learning techniques [14]. Our focus is to use deep learning-based techniques for bugs classification to achieve improved performance of bugs classification. For this, we have used four deep learning-based techniques for bugs classification.

To address the issue of bug classification (henceforth denoted as BC), we explore performing a contextual analysis of the bug resolution report. We have collected a dataset consisting of approximately 1.36 million bug resolution reports through the Bugzilla interface of eight open source projects: Apache, Eclipse, KDE, LibreOffice, Linux, Mozilla, NetBeans, and OpenOffice. We classify bugs into eight categories: database, enhancement, infrastructure, logic, networking, performance, security, and usability. These bug categories have been identified based on the results of a survey of eight experienced programmers. To classify the bugs reported for the considered open source projects into the

^{*} Corresponding author.

E-mail addresses: jpmehar.iitkgp@gmail.com (J.P. Meher), bsws.sourav@gmail.com (S. Biswas), rajib@cse.iitkgp.ac.in (R. Mall).

Bug 573743 - The Eclipse Security Mailing list is publicly accessible!

Status: RESOLVED FIXED

Alias: None

Product: Community

Component: Vulnerability Reports ([show other bugs](#))

Version: unspecified 

Hardware: All All

Importance: P1 critical ([vote](#))

Target Milestone: --- 

Assignee: Eclipse Webmaster  

QA Contact:

URL:

Whiteboard:

Keywords: security

Depends on:

Blocks:

Reported: 2021-05-25 04:16 EDT by Peter Stăckli 

Modified: 2021-05-25 11:27 EDT ([History](#))

CC List: 1 user ([show](#))

See Also:

Fig. 1. A sample bug report from Eclipse.

identified taxonomy, we have used word mover distance [15] algorithm, as it considers the semantics of the words present in the bug resolution summary reports. To this end, we use four dot-product-based attention [16] models (Transformer, BERT, CodeBERT, and DistilBERT) to classify the bugs. Our experimental results show that the BERT-based technique outperformed the other three machine learning based techniques [6,17,18]. Our experimental results show that our BERT-based technique achieved on an average 16.88% improvement in F1-score over the existing techniques. We now summarize the important research contributions of our work in the following:

- We have developed a heuristic approach that annotates bug reports based on their summary using an earth mover distance-based technique.
- We have proposed four neural approaches to automatically classify the annotated bugs using a self-attention-based transfer learning mechanism.
- We have made available a moderately large corpus of annotated bugs that can be used as a benchmark dataset by researchers working on various software engineering problems.

1.1. A motivating example

For a better understanding of a bug report, let us consider a recently (i.e., May 25, 2021) raised bug report in the Bugzilla platform for the Eclipse project, as shown in Fig. 1. It is evident from the bug summary that it essentially describes a security vulnerability: the security mailing list is accessible to the public. In addition to this, it contains a rich source of information such as the status of the bug, alias, product, report date, modification date, cc list, component, version, hardware, importance, target milestone, assignee, QA contact, URL, whiteboard, keywords, and blocks. A bug summary can provide important information about the bug. Although it is not true for all the cases, as some bug summaries are not clear, the bug resolution summary of most of the bug reports is clear and complete. We are therefore encouraged to consider only the bug resolution summary in our study.

Paper Structure: The rest of the paper is organized as follows: Section 2 presents an overview of the basic concepts used in our approach. In Section 3, we survey the related work on bug classification. Section 4 discusses our proposed approach. In Section 5, we report the results of our experimental studies. Section 6 discusses the analysis of the results. Possible threats to our experimental outcomes are highlighted in Section 7. Section 8 concludes this paper.

2. Basic concept

In this section, we provide a brief overview of few important concepts on which our work is based.

2.1. Word mover's distance (WMD)

WMD method measures the dissimilarity between two text documents [19]. Each word is projected into a higher-dimensional embedding space, and the words are represented so that semantically similar words are positioned in proximity to other words.

To measure the similarity between two text documents, it uses a vector representation of the two documents. This vector representation is obtained by word2vec [19] embedding. After the words are embedded into vectors, the semantic distance among them is computed. Similarity estimation is a three-step approach: Text representation, Similarity metric computation, and Flow matrix computation. We explain these in the following.

2.1.1. Input text representation

Input text is represented as a vector using a bag of words representation denoted as d , in which each element represents the normalized frequency of a word i :

$$d = [d_1, d_2, \dots, d_n]^T \quad (1)$$

where, $d_i = \frac{c_i}{\sum_{j=1}^n c_j}$ and $c_i = \{\text{word } i \text{ appears } c_i \text{ times in a sentence}\}$. The main idea used in this approach is estimation of the minimum distance between two words from different documents or sentences.

2.1.2. Similarity definition

Once the vectors are represented in a higher dimensional embedding, the traveling cost between two words is computed in terms of the Euclidean distance among them. The similarity distance between two vectors x_i and x_j is represented as:

$$c(i, j) = \|x_i - x_j\| \quad (2)$$

where, $c(i, j)$ is the traveling cost from x_i to x_j .

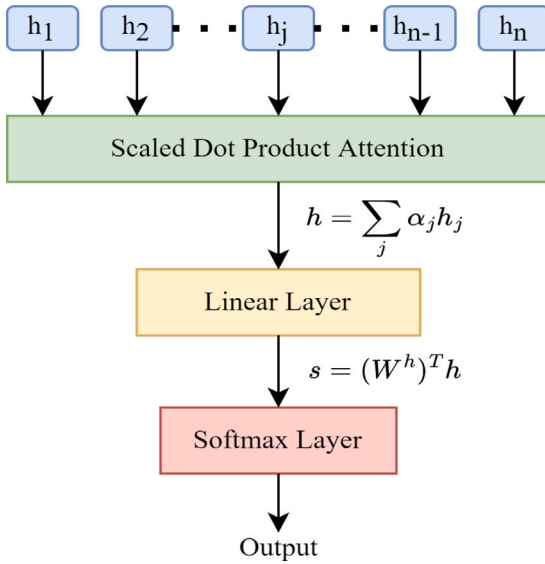


Fig. 2. Multi-class classification with attention.

2.1.3. Flow representation

A flow matrix, $T \in R^{n \times n}$ represents how many times $x_i \in d$ transforms into $x_j \in d'$, where $T_{ij} \geq 0$ represents the number of words from document d that are transformed into document d' . T_{ij} is represented as follows.

$\sum_j T_{ij} = d_i$ and $\sum_i T_{ij} = d'_j$, where d_i is the sum of all outgoing flows from the word i and d'_j is the sum of all incoming flows to the word j . Further, the distance between d and d' is computed in terms of minimum cumulative cost to transfer all words from d to d' i.e., $\sum_{i,j} T_{ij} c(i, j)$. The minimum distance is obtained by solving the optimization formulated as Eq. (3):

$$\begin{aligned} \min_{T \geq 0} \quad & \sum_{i,j=1}^n T_{ij} c(i, j) \\ \text{subject to:} \quad & \sum_{j=1}^n T_{ij} = d_i, \quad \forall i \\ & \sum_{i=1}^n T_{ij} = d'_j, \quad \forall j \end{aligned} \quad (3)$$

2.2. Attention-based classification

Given the summary of a bug (henceforth denoted by S_b), and a set of possible predefined bug classes K (in this work, $k_i \in \{0, 1, \dots, 7\}$), we are interested in estimating the probability of the most suitable class k_i with S_b . Here, S_b is a natural sequence of N linguistically correct tokens such that, $S_b = \{t_i\}_{i=1}^N$, where $N = |S_b|$ (see Fig. 2).

Among the available attention mechanisms reported in the literature, possibly the most commonly used one is the scaled dot-product attention [16]. In this attention, two scores are estimated: *polarity* and *attention*. Among these, *attention score* indicates the degree of the token's significance on global assessment (e.g., classification, generation, etc.). Therefore, an attention score with a scaled dot-product is more appropriate for multi-class bug classification. Consider the input sequence of S_b of length n be: $S_b = \{t_1, t_2, \dots, t_j, \dots, t_{n-1}, t_n\}$, where t_j is the j th token in the input, which is represented before the attention layer as h_j . $h_j \in \mathbb{R}^d$, where d represents the input dimension of h . The attention score for the j th token is estimated as:

$$a_j = \frac{h_j^T V}{\Lambda} \quad (4)$$

In Eq. (4), Λ is the scaling factor which is typically set to $\sqrt{d}$ [16], and $V \in \mathbb{R}^d$ is the context vector that can be treated as a fixed query asking for the “most significant word” from S_b . This is used to assign the most appropriate class to S_b . The token representation h_j can be the output of an encoder or word embedding. The attention weight corresponding to h_j can now be calculated as:

$$\alpha_j = \frac{\exp(a_j)}{\sum_{j'} \exp(a_{j'})} \quad (5)$$

In the next step, the whole input sequence and the output of the linear layer can be represented by Eqs. (6) and (7) respectively.

$$h = \sum_j \alpha_j h_j \quad (6)$$

$$s = (W^h)^T h + b_h \quad (7)$$

where, s is the *popularity score* based on instance level, and b_h is the associated bias. $W^h \in \mathbb{R}^{d \times K}$ is the weight matrix of the final linear layer, and h is the representation of the input instance. Finally, the probability distribution of k_i , $\forall i$ is passed through a softmax layer as:

$$P = \text{Softmax}(s) \quad (8)$$

3. Literature review

Bug classification is a well-investigated area in the software engineering research domain [12]. Several techniques have been proposed to classify bug reports, including text-based [20], fuzzy logic-based [10], text mining approach [21], automated approach [22–24], association mining [9], and machine learning [7,8,10] approaches. We discuss different taxonomies available in the literature for bug classification. Subsequently, we only discuss different machine learning (ML) based bug classification techniques as these are directly comparable to our work.

Chillarege et al. [25] were probably the first to classify program defects. They introduced a taxonomy comprising 13 root causes and named it *Orthogonal Defect Classification (ODC)*. ODC primarily focuses on causal analysis of program structure to fix a bug rather than basing it on bug type.

Tan et al. [26] reported in their study that better bug classification requires a deep understanding of the bug characteristics. They manually analyzed 2060 bugs from three (Linux, Mozilla, and Apache) open-source projects. In a similar direction, Zheng et al. [27] attempted an empirical understanding of bug characteristics available in the open-source cloud platform. Recently, Wu et al. [28] investigated data quality of security bug reports for predictive modeling. They claimed that proper labeling of the bug reports substantially boosts classification performance.

Zimmermann et al. [5] investigated the quality of bug reports by interviewing programmers to find out what information was most important to them in bug reports. In addition, they developed a machine learning-based model (i.e., CUEZILLA) that could rate the quality of reported bugs on a scale from one to five (1: very bad and 5: very good). Fan et al. [29] used machine learning techniques to distinguish valid bugs from invalids on five open-source projects: Eclipse, Mozilla, Firefox, Thunderbird, and NetBeans. They reported their classifier performance to be 0.81 in terms of AUC.

With the advancement to neural computing, many researchers have attempted to classify bugs using machine learning-based techniques [7–11,30,31]. Alazzam et al. [11] used a graph-based technique to classify Eclipse bugs into different priorities. Similarly, Chawla et al. [10] suggested a method for automated bug labeling using the existing linguistic information in the bug reports. As the initial step, they used a standard NLP preprocessing technique to preprocess the bug report. Subsequently, they used Term Frequency–Inverse Document Frequency (TF–IDF) [32] technique to identify semantically similar phrases to cluster bugs of similar classes. Their empirical results indicated a 61.5%

Table 1
A Summary of important related work.

Sl No	Author	Classification technique	#Class	#Project	Dataset size	Results
1	Tan et al. [26]	Manual analysis	4	3	2060	75.67% F1-score
2	Zimmermann et al. [5]	ML	–	3	289	50% Accuracy
3	Fan et al. [29]	ML	2	5	560 697	81% AUC
4	Chawla et al. [10]	TF-IDF	2	3	5590	79.1% F1-score
5	Behl et al. [33]	NB	2	3	10 000	93.1% Accuracy
6	Otoom et al. [23]	NB, SVM, RF	2	3	5800	94.23% Accuracy
7	Catolino et al. [6]	NB, SVM, LR, RF	9	3	1280	74% AUC
8	Ye et al. [17]	LSTM	2	3	9000	86.5% F1-score
9	Tang et al. [18]	Attention + LDA	2	3	17 054	73% F1-score

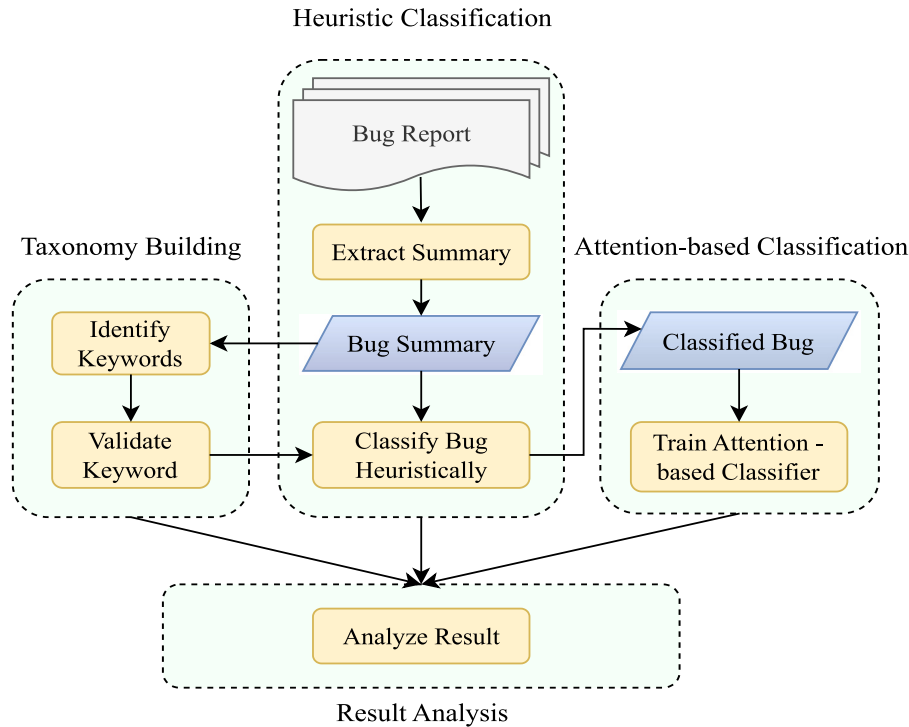


Fig. 3. A Schematic representation of our proposed approach.

accuracy in classifying polish bugs and a 62.8% accuracy for security bugs.

In addition to TF-IDF, Behl et al. [33] used a bayesian classifier to distinguish security bugs from others. Their experimental results claimed that TF-IDF is marginally better than the Bayesian classifier. Later, Otoom et al. [23] used three ML-based classifiers, namely Naive Bayes, Random Forest, and Support Vector Machine (SVM). They experimented with defects of three open-source projects: SWT, AspectJ, and Tomcat. They claimed that SVM outperformed other classifiers and resulted in 93.1% accuracy. Similarly, Catolino et al. [6] performed a comprehensive analysis of bug characteristics of 1280 bugs to classify them into nine categories using ML-based techniques. Toward this, they used Naive Bayes, SVM, Logistic Regression, and Random Forest classifiers. Their empirical results claimed a 74% AUC when using SVM. Tang et al. [18] proposed an attention-based technique with Latent Dirichlet Allocation (LDA) to classify 2230 bugs into two categories (Bohrbug and Mandelbug). The experimented using available dataset. Their experimental results were evaluated using F1-score and it was found to be 73% when attention and LDA are combined. We summarize the important related work in the Table 1.

From Table 1, it can be observed that researchers have attempted a variety of classification techniques, ranging from manual analysis to deep learning-based techniques. However, superior classification performance (in terms of F1-score) can be observed for deep learning-based technique [17]. Further, it can be observed from Table 1 that

the reported techniques were experimentally studied with a rather limited number of projects amounting to a rather small number of bug resolution reports.

4. Our proposed approach

We present an automated approach to classify software bugs. Our approach uses deep learning techniques. We hypothesize that our technique should exhibit improved learning from the contextual features of the bug resolution summary report due to the use of the attention mechanism. This should lead to boosting the performance of the classifier. We schematically represent our proposed approach in Fig. 3. As can be seen from Fig. 3, our approach is divided into three parts: (i) building a bug taxonomy based on the important keywords present in the bug summary, (ii) labeling the bug summary for every bug based on a heuristic-based technique, and (iii) classifying the labeled bugs using an attention-based classification technique. We discuss each of the parts of our proposed approach as follows.

4.1. Taxonomy building

To characterize and classify different classes of software bugs, we construct a bug taxonomy by adopting an iterative procedure of content analysis on bug summary with a different objective at each sprint. For this, we identified eight bug categories: *database*, *enhancement*,

Table 2
Keywords for different classes.

Sl. No	Class	Keywords
1	Database	abort, transaction, dump, store, retrieve, sql, leak, bound, find, delete, insert, buffer, external, internal, size, stack, queue, data, database, storage, inconsistency, loss, overwritten, duplicate, missing, update, lock, recover, access, <i>table, commit, abort, synchronize, timeout, reset</i>
2	Enhancement	remove, delete, move, clean, refactor, rename, content, rid, unit, base, content, resolve, polish, text, miss, tab, bookmark, drop, manage, typo, improve, enhance, modify, change, add, adapt, refine, port, support, upgrade, migrate, replace, update, delete, <i>image, picture, line, alert</i>
3	Infrastructure	environment, configuration, delete, kernel, miss, system, deploy, build, capacity, order, working, duplicate, recognized, respond, switch on, power on, broken, crash, memory, USB, drive, RAM, cache, <i>operating system, OS, unavailable, shut down, reboot, release</i>
4	Logic	logic, delete, incorrect, invalid, wrong, calculate, output, error, expected, fail, stack, queue, miss, block, major, result, accuracy, mistake, deviation, terminate, invalid, loop, decision, functionality, algorithm, crash, input, data, condition, branch, <i>random, timeout, reset, null, cancel, exception</i>
5	Networking	network, client, server, url, html, http, https, packet, route, acknowledgement, delete, interface, miss, card, node, invalid, noise, switch, signal, peer, LAN, port, MAC, address, duplicate, layer, protocol, session, synchronous, asynchronous, carrier, ethernet, api, connect, bandwidth, DNS, address, jitter, <i>firewall, traffic, entry, exit</i>
6	Performance	response, latency, delay, performance, time, miss, slow, cpu, lag, long, invalid, hang, reduce, time, fast, vary, execution, run, time-out, time out, lazy, render, buffering, Shuttering, broken, block, stop, intermittent, freeze, jitter, <i>throughput, vary</i>
7	Security	encryption, secure, corrupt, privacy, password, miss, delete, access, blocker, audit, sandbox, bypass, vulnerable, safe, intrude, password, password-manager, protect, authenticate, compromise, access, stolen, denial, prevent, broken, <i>firewall, key, public, private, threat</i>
8	Usability	window, menu, zoom, use, click, cursor, keyboard, select, mouse, feature, hitting, entering, bar, button, icon, engine, scroll, color, image, look, feel, brightness, sound, resolution, drop-down, navigation, contrast, frozen, readability, legibility, inconsistency, disable, enable, invisible, invalid, miss, popup, form, check box, delete, combo box, <i>radio button</i>

infrastructure, logic, networking, performance, security, usability. During taxonomy building, if none of the keywords belong to any of the categories, we annotate it as *Out of Class (OC)*. Similarly, if the bug summary is not adequately documented, not reproducible, or a suggestion, it is considered *Not a Bug (NB)*. If a reported bug is NB or OC, we do not consider it in our study, since we are interested in the textual content rather than debugging or fixing it. This helps in achieving better generalizability of the proposed model. We identify keywords for each bug category as shown in Table 2 and define each bug category as follows:

- **Database:** This first category primarily focuses on a bug related to accessing a database. It causes an unanticipated system breakdown, leading to irrecoverable data loss. Most of these bugs require retrieving, adding, deleting, and modifying the data stored in a database. For example, the following bug describes a null pointer exception that occurred while trying to store a reference to the session in the context and subsequently accessing it in the subsequent request.
Bug Summary: “*Problem when I try to store a reference to the session in the context and then when I try to access it in the next request I get NullPointerException BugRat Report#739*”,
Apache - Tomcat3: [Bug ID - 429](#)
In the above bug summary, it can be observed that the summary contains important keywords such as *store, access* etc. It can also be observed from Table 2 that the above mentioned keywords are matching with the keywords of the Database class. Therefore, our model classifies this bug summary as a database bug. Similarly, the same technique is applied in the following bug categories.
- **Enhancement:** These are possibly the most commonly occurring bugs. A bug of this type relates to minor improvement and modification to the system. In the following, we provide a sample bug summary below to illustrate an issue flagged. It causes the source files to be deleted if another application locks the project folder.
Bug Summary: “*[resources] renaming project to the same name but with different case causes source files to be deleted if project’s folder is locked by other application [refactoring]?*”
Eclipse - Platform: [Bug ID - 25457](#)

- **Infrastructure:** The bugs pertain to environment, installation, configuration, and so on fall under this category. We present an illustrative example below extracted from KDE - Kwin project. It indicates that third-party apps do not support international characters: the application crashes when these are entered.
Bug Summary: “*International characters (åäö) doesn’t work in third party apps (Applix menus and xmms song list) Applications crash if these characters are inserted.*”
KDE - Kwin: [Bug ID - 27460](#)
- **Logic:** A bug raised due to any problem in logical or algorithmic implementation is regarded as a logic bug. For further information, an example collected from LibreOffice project highlights an update issue where it cannot update a formula on DEI or BS. The textual summary of the bug is represented as follows:
Bug Summary: “*EDITING: Calculate formula: DEI or BS to empty a cell with a formula, does not result in updating a formula referencing that cell*”
LibreOffice: [Bug ID - 56278](#)
- **Networking:** Bugs pertaining to network connection problems or server issues fall under this category. The possible reasons for this kind of bug are problems in the network server, invalid/incorrect communication protocol followed, and so on. For instance, a bug raised in Linux project is shown as follows, highlighting a segmentation fault in the program.
Bug Summary: “*Problems with 100% CPU usage when using Skype on VT1708/A [Azalia HDAC] (VIA High Definition Audio Controller) (rev 10) with AD1984 A codec*”,
Linux - Drivers: [Bug ID - 14932](#)
- **Performance:** This category of bugs is concerned about the *responsiveness* of the system. Essentially, a performance bug is raised when the system’s performance is not acceptable. For example, the following bug summary reports that the list containing consecutive list items with only white space renders the list items without advancing the vertical position.
Bug Summary: “*Lists which contain consecutive list items which contain only whitespace (i.e., empty) render the list items without advancing the vertical position.*”,
Mozilla - Core: [Bug ID-179596](#)

- **Security:** The bugs concerning security vulnerabilities fall under this category. Usually, any unauthorized access to the system or unintended user trying to access confidential or sensitive data are considered to be a security bug. In the following, we give an example of a security bug. It is collected from *NetBeans* project. It reports that when trying to click the right button for creating getter() and setter(), the menu is unavailable, resulting in a block. Bug Summary: “org.netbeans.modules.javacore.jmiimpl.javamodel.GeneralException: No record exists with key D6E85CF2-9B76-11DB-B789-64DCE471AA77:000000000000289D.”, NetBeans - Java: [Bug ID - 92263](#)
- **Usability:** This category of bugs pertain to the user interface. In the following, we provide an example of a bug summary from *Open Office* project that describes an issue related to the product’s user interface. It can be seen that the font work dialogue box incorrectly displays the shadow when applied to a text that is gone through a baseline manipulation. Bug Summary: “The vertical shadow utility in the fontwork dialogue box incorrectly displays the shadow when applied to text that underwent a baseline manipulation.”, OpenOffice - Impress: [Bug ID-22749](#)

Based on the description of the above bug categories and their key terms, two software engineering researchers (the first, and second authors), both having at least four years of experience in software development, manually classified a set of 200 bugs randomly sampled from the collected bugs. Henceforth, we denote them as *content analyst*. Their primary task is to understand the bug’s context and analyze the textual features contained therein, to assign an appropriate bug class to each bug. Essentially, they try to identify and analyze the key terms present in the bug summary that contribute to the nominated category. Finally, two reviewers verified all the generated keywords. In the following, we elaborate each sprint to building a bug taxonomy and the validation of the resulting taxonomy.

Sprint 1: The primary objective of the first sprint is to identify common keywords of different classes. To this end, 200 bug summaries were randomly sampled, and their textual features were analyzed to identify discriminative keywords from their summaries. Afterward, the *content analysts* stereotyped the keywords based on the definition and their prior knowledge. During this process, the bugs containing ambiguous summaries (keywords belonging to multiple categories or do not belong to any of the categories) are kept aside and marked *unclassified*. Afterward, a discussion was opened between the content analysts to (i) verify the keywords extracted from sampled bugs, (ii) further discuss the *unclassified* sampled bugs, and (iii) make the outcome a reference for subsequent sprints. The disagreement in this sprint and the ambiguous bugs are discarded, and the remaining constitutes a draft taxonomy for subsequent sprints.

Sprint 2: In this sprint, the draft taxonomy resulting from sprint 1 is used as a reference for classifying another 200 bug reports. Two advantages are expected of this. It can help in assessing the validity and correctness of the classes resulting from sprint 1. It can also help in discovering unanticipated keywords related to different bugs. The *content analysts* tried to classify another set of 200 randomly sampled bug summaries. Again, to reach a consensus, they opened a discussion on disagreements between them to refine the draft taxonomy. As a result, the draft taxonomy is improved with more keywords in each category.

Sprint 3: Gathering all the taxonomic information from *sprint1* and *sprint2*, the content analysts further extracted keywords from another 400 bug summaries and annotated them. It is worth noting that they reached a consensus that no more keywords are being extracted. It is possible that some of the keywords map to multiple bug categories. For example, the keyword delete could possibly map to any of the bug categories: database, enhancement, infrastructure, logic, or networking depending on the context of usage in the bug resolution summary. This

Table 3

Industrial expert details.

S.N.	Niche area	Year of expertise	Industry
1	Database	13	TCS Ltd.
2	Enhancement	9	TCS Ltd.
3	Infrastructure	15	TCS Ltd.
4	Usability	13	TCS Ltd.
5	Logic	14	TCS Ltd.
6	Networking	12	LinkedIn Corp.
7	Performance	9.5	Accenture plc.
8	Security	11.5	Apple Inc.
–	Average	12.12	–

makes a case for a further round (sprint 4) of discussions among the content analysts.

Sprint 4: This is the final round of the taxonomy building phase. The main objective of this round is to identify the overlapping keywords: keywords that belong to multiple categories. To this end, a random sample of 320 bugs were selected from 800 (from sprint 1, 2, and 3) bug summaries. After a careful analysis, a few existing keywords were flagged as overlapping. The content analysts then included these keywords in the respective categories. As a result, a more comprehensive taxonomy was obtained. The refined keywords are shown in [Table 2](#).

4.1.1. Taxonomy validation

To ensure the validity and correctness of our proposed taxonomy, we use an expert verification technique. For this, we took the help of experts. We decide to keep their identity anonymous on account of privacy concerns. However, we provide the necessary information in [Table 3](#). The experts were contacted through the personal contacts of the first author. Subsequently, individual categories and keywords are mailed to the experts. We did not share the definitions of the bug category, to prevent any bias. The experts are requested to ensure the correctness and validity of the identified keywords by:

- examining the occurrence of keywords in bug summary of the given category.
- suggesting if any keyword that needs to be removed, added, or modified.

4.2. Heuristic-based classification/estimating ground truth

Since the collected bug summaries do not have labeled bug categories, we initially follow an empirically proven heuristic-based approach to estimate the ground truths for the collected bugs. In particular, we use WMD [15] technique to estimate bug summary similarity for classification. For this, we introduce an algorithm.

Algorithm 1 depicts our heuristic-based bug annotation. The algorithm takes input a list of bug summaries and a list of bug classes with the corresponding class keywords and outputs the class of each bug summary. The bug summary list contains the pre-processed textual representation of the bug summary. In Step 1, the total traveling distance from a bug summary to a bug class (please refer Eq. (3)) is initialized to zero. Step 2 to 10, calculate the word mover distance for each bug summary with the keywords of each class and classify the bug summary to a particular class based on the calculated distance. A bug summary is assigned to a class for which the estimated distance is minimum. Finally, in Step 10, the class of the bug summary is returned. We consider eight bug classes as already discussed. So, the time complexity of our algorithm is $\mathcal{O}(m * n * p) \cong \mathcal{O}(n^3)$. Here, m is the number of summary reports examined, n is the number of words in a summary, and p is the number of keywords in a bug class.

Algorithm 1: HEURISTIC-BASED BUG ANNOTATION

Input: $Summary[[]]$, $Class[[]]$ // *Summary* and *Class* are two list of lists containers which contain bug summaries and keywords of bug classes respectively

Output: $index[Class]$ // Bug class of a particular Bug summary

```

1 Initialization:  $dist_{total}[[]] \leftarrow 0$ 
2 for  $i = 1$  to  $|Summary[[]]|$  do
3   for  $j = 1$  to  $|Class[[]]|$  do
4     for  $k = 1$  to  $|Summary[i]|$  do
5        $dist[i][k] \leftarrow \infty$ 
6       for  $l = 1$  to  $|Class[j]|$  do
7          $dist_{wmd} = WMD(Summary[i][k], Class[j][l])$ 
8         if  $(dist_{wmd} \leq dist[i][k])$  then
9            $dist[i][k] = dist_{wmd}$ 
10         $dist_{total}[i][j] = dist[i][k] + dist_{total}[i][j]$ 
11       $index = index(Min(dist_{total}))$ 
12 return index

```

4.3. Attention-based classification technique

We provide an overview of different attention-based classification techniques used in our experimentation. We first discuss the vanilla transformers-based classification technique. Subsequently, we discuss using CodeBERT [34] in bug classification. Afterward, we used a light version of BERT (DistilBERT [35]) architecture, and finally, we used BERT for bug classification.

4.3.1. Transformer

To capture context-aware input encoding of bug summary, we trained a vanilla transformer-based architecture [16]. We have selected this architecture in our study, as it attends to important words present in the input sequence to classify bug summary.

4.3.2. CodeBERT

CodeBERT [34] can capture the bi-modal representation of both programming language and natural language. In our case, a bug summary may contain terminologies related to the programming language and the natural language as it describes the failure of a system. Therefore, we thought to consider CodeBERT in our experimentation.

4.3.3. BERT

It is a transformer-based model that captures the statistical relationships among the words present in an input sequence. It subsequently learns the contextual input representation over multiple iterations. Since BERT [36] is pre-trained on an unsupervised manner and can be effectively used for transfer learning [37] for any supervised task, we feel encouraged to include this model in our experimentation.

4.3.4. DistilBERT

DistilBERT [35] (we denote as DB) is a lightweight optimized version of BERT [36] that preserves approximately 97% of the original BERT. Since it is a lighter, smaller, and faster version of BERT, we feel encouraged to consider this model in our experimentation.

5. Experimental studies

In this section, we first discuss the data collection technique used. We then elaborate on the iterative process for taxonomy building. Subsequently, we describe the preprocessing technique used to preprocess the bug summary. Further, we provide an overview of our

Table 4

Mining details of the Corpus.

S.N.	Project	From	# of bug
1	Apache	01-01-2000	91 273
2	Eclipse	07-11-2001	320 576
3	KDE	01-01-2000	202 807
4	LibreOffice	25-01-2011	47 511
5	Linux	01-01-2000	14 776
6	Mozilla	23-09-2002	502 989
7	NetBeans	01-06-2000	136 539
8	OpenOffice	01-05-2002	41 506
–	Total	–	1 357 977

experimental setup, followed by the classification techniques used in our experimentation. Finally, we discuss the evaluation metrics used to measure the performance of our proposed technique.

5.1. Data collection

While analyzing and classifying bugs in proprietary is restricted on account of intellectual property rights, bugs of many open source projects are logged through online bug tracking systems such as *Bugzilla* and made publicly available. We take this advantage and feel motivated to analyze and classify bugs in open-source projects. To this end, we selected eight projects, the details of which are presented in Table 4. We select all the valid bugs raised so far (till 31st July 2022) from the project's initial release. To ensure this, we consider the bugs with status *confirmed*, *assigned*, *reopened*, *resolved*, *verified*, and *closed*, and resolution *fixed*, *not a bug*. As a result, we gather 1357977 valid bugs for further analysis and classification.

5.1.1. Independent and dependent variables:

The objective of our study was to classify bugs based on the information contained in corresponding bug reports. We used the bug summary present in bug reports as an independent variable in our bug classification model. This decision was made in light of reported findings that reported that bug summary messages in bug reports are accurate descriptions of the bug in practice, making them a rich source of information for classifying bugs [5]. On the other hand, the dependent variable is the bug class. Our empirical study has focused on eight predefined classes (e.g., database, enhancement, infrastructure, logic, networking, performance, security, and usability).

5.2. Data preprocessing

As bug summary mainly contains natural language sentences, we apply standard NLP techniques such as *tokenization*, *stop word removal*, and *lemmatization* to preprocess each bug summary. In the following, we have discussed the data preprocessing steps:

- First, we have perform tokenization of the bug summary. Here, we substitutes each word in a sentence with its corresponding token. In this process, the words in a sentence are separated into linguistically meaningful elements. Subsequently, all the punctuation marks (., ,, !, etc.) are removed, and the alphanumeric symbols (*, '@', '#', '\$', etc.) appear in the sentence are dropped and replaced with empty spaces. As an example, we illustrate how the performance bug summary presented in 4.1 is converted into its tokenized form as follows:
 “list”, “which”, “contain”, “consecutive”, “list”, “item”, “which”, “contain”, “only”, “white”, “space”, “empty”, “render”, “the” “list”, “item”, “without”, “advance”, “the”, “vertical”, “position”.
- Next, we apply the stop word removal technique on the tokenized sentence. Here, the stop words basically cover the *article*, *helping verb*, *modal verb*, *pronoun* and many more. The basic intuition in removing these stop words is that even tough these are the most

Table 5
Bug distribution of bug classes for each project after annotation.

S.N	Class	Project								Total
		Apache	Eclipse	KDE	LibreOffice	Linux	Mozilla	NetBeans	OpenOffice	
1	Database	9749	16 264	14 502	5304	934	31 845	11 260	4594	94 452
2	Enhancement	20 135	151 928	33 666	12 296	729	148 353	28 426	9718	405 251
3	Infrastructure	9879	86 267	29 892	4336	4964	55 323	16 318	4980	211 959
4	Logic	15 598	37 656	28 089	6543	2274	55 651	28 323	6297	180 431
5	Network	13 075	2131	27 827	5081	2900	84 589	23 557	6753	165 913
6	Performance	4374	17 435	12 363	1868	1306	35 834	7103	1559	81 842
7	Security	2817	1443	6170	998	482	25 602	3678	1714	42 904
8	Usability	15 646	7452	50 298	11 085	1187	65 792	17 874	5891	175 225
–	Total	91 273	320 576	202 807	47 511	14 776	502 989	136 539	41 506	1 357 977

frequently used words in English language, they do not contribute to the statistical estimation of language modeling for downstream tasks [38]. Therefore, removing these words may improves the performance of the model.

- Finally, in the preprocessing step, we use the lemmatization technique on the tokenized sentence. In this step, each word is essentially converted into its canonical form. For example, the word ‘advancing’ can be lemmatized into its canonical form ‘advance’. Other related words, such as ‘advanced’ can also be converted into the same root word. The main intuition behind this step is that it groups all the related words with the same root word. This step in NLP makes it easier for prediction and generation tasks.

5.3. Experimental setup

The training environment we have used is the supercomputer **PARAM Shakti** which is composed of multiple heterogeneous and hybrid clusters of Intel Xeon Skylake processors and NVIDIA Tesla V100 computing resources. It runs on Linux – CentOS 7.6 operating system. This workstation uses the Lustre parallel file system. It comprises 2 Primary nodes, 8 User account nodes, 10 Different Service nodes, and 442 (CPU+GPU) nodes, with a total peak computational capacity of 1.66 (CPU+GPU) PFLOPS. The workstation is built using Intel Xeon SKL G-6148 and NVIDIA Tesla V100 processors, with a total optimal efficiency of 1.6 PFLOPS. The cluster is made up of computing nodes that are connected by an InfiniBand interconnection network from Mellanox (EDR).

We followed an approach similar to that used by Gou et al. [39] to tune the hyperparameters. We conducted experiments using different combinations of hyperparameters for identifying the optimal combination. We tested with three combinations of batch sizes: 32, 64, and 128. For the learning rate, we tried: $1e^{-3}$, $1e^{-4}$, and $1e^{-5}$. Similarly, we ran our experiment for different numbers of epoch: 10, 20, 30, and 40. We found that in the optimal combination of batch size, learning rate and epoch need to be set to 64, $1e^{-4}$, and 20 respectively. Therefore, we fixed the batch size to 64. The learning rate is set to $1e^{-4}$. The training cycle is run for 20 epochs as we observed that all model saturates within this. The maximum length of the bug summary is set to 256, as the length of all the summaries lies under this range. To govern the learning momentum, we set all hyperparameters identical for all the techniques discussed in Section 4.3 for a fairer comparison.

Two cross validation techniques are popular: K-fold cross validation, and Leave-one-out cross validation. However, we observe from our experiments that Leave-one-out cross validation outperforms K-fold cross validation technique. A possible reason for this is the limited size of the dataset that we have used. Consequently, we report the results of only the Leave-one-out cross validation technique from our experimental study. We used standard required libraries such as pandas, ktrain, gensim, etc., using the PyTorch framework.

5.4. Evaluation metrics

The task of bug classification used in our work can be formalized as a task of multi-class text classification. The widely preferred evaluation metrics for this task are *F1-measure* [40], and *AUC-ROC* [41]. We, therefore, feel motivated to use these metrics to measure the performance of our classification task for comparison with the related work.

The F1 measure is a composite metric that estimates the harmonic mean of precision and recall. It is a measure of the model’s accuracy over a given dataset. Since it accounts for both the precision and recall in estimating the classifier’s performance, it is considered to be a robust metric in estimating predictive performance. This metric is generally represented in terms of F1-score as calculated as:

$$F1\text{-score} = \frac{TP}{TP + \frac{1}{2} * (FP + FN)} \quad (9)$$

Where, TP denotes true positive, FP stands for false positive, and FN represents false negative in the classifier output.

In the second, we use AUC-ROC curve [41] to interpret the classifier performance at all threshold points. This curve essentially plots two parameters: true positive rate and false positive rate. True positive is basically the recall ($TP/(TP+FN)$) of the classifier, and the false positive rate is calculated as $FP/(FP + TN)$. This metric quantifies how well a classification model can distinguish between the various classes. The closer the AUC-ROC to 1, the better the classifier performance. On the other hand, the closer the AUC-ROC to 0.5, the poorer the model’s ability.

While F1-measure and AUC-ROC are two well-accepted evaluation metrics, they do not consider all the quadrants of a confusion matrix viz. true positive (TP), true negative (TN), false positive (FP), and false negative (FN). Consequently, these can result in inflated results, when the dataset is unbalanced. To address this issue, we have experimented using Matthews correlation coefficient (MCC) [42]. MCC considers all the four quadrants. It is defined as follows:

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP) * (TP + FN) * (TN + FP) * (TN + FN)}} \quad (10)$$

MCC metric evaluates to a value in the range $[-1, +1]$. A value of -1 indicates a mismatch between prediction and observation. When MCC evaluates to 0, it shows that the model performs no better than a random model. An exact match is indicated when MCC evaluates to $+1$.

6. Results analysis

In this section, we report the empirical results from our experimentation. We first present both quantitative and qualitative overview of the performance of our proposed classification technique. Subsequently, we quantitatively compare the performance of our approach with existing techniques using F1-Score.

Table 5 outlines the distribution of bugs over the eight classes across eight projects considered in our study. It is evident from Table 5 that

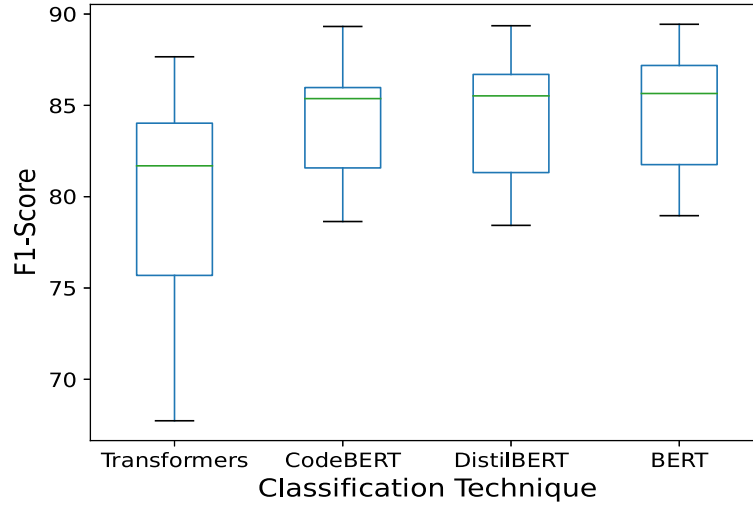


Fig. 4. Box plot representation of performance.

Table 6

F1-Scores of different BC techniques (in %).

Sl No.	Project	Transformers	CodeBERT	DistilBERT	BERT
1	Apache	80.31	85.46	85.35	85.46
2	Eclipse	87.66	89.32	89.36	89.44
3	KDE	83.14	85.81	85.69	85.84
4	LibreOffice	75.96	81.44	81.29	81.5
5	Linux	67.73	78.64	78.43	78.96
6	Mozilla	86.68	85.28	88.15	88.43
7	NetBeans	83.07	86.47	86.21	86.77
8	OpenOffice	74.88	81.62	81.33	81.84
–	Average	79.93	84.25	84.48	84.78

relatively large number of bugs are raised for Mozilla project followed by Eclipse. On the other hand, less number of bugs are raised for Linux projects. Similarly, enhancement bugs accrued the highest number of bugs across bug classes, followed by infrastructure-related bugs, and the least number of bugs are raised in the security class.

6.1. Performance of our proposed technique

Table 6 presents the F1-score of our four proposed attention-based bug classification techniques for eight open-source projects. From a high level point of view, it can be confirmed that all the four techniques perform differently, across different projects. Looking more in depth, it can be observed from Table 6 that the BERT-based BC technique achieved the highest F1-score (89.44) among all the techniques for the Mozilla project. On the other hand, the Transformer-based BC technique showed the lowest F1-score (67.73) among all, for the Linux project. From Table 6, it can be estimated that the F1-score achieved by our four BC techniques is highest (89.44) for Eclipse project followed by Mozilla (88.43) project. On the other hand, it is estimated to be the lowest (78.96) for Linux project. Therefore, it can be hypothesized from Tables 5 and 6 that the BC technique performs better when the number of bugs increases.

Fig. 4 presents the box plot representation of the performance of our proposed techniques. Here, X-axis represents BC techniques, whereas Y-axis represents F1-score. It can be observed from the figure that among the four proposed techniques, the BERT-based bug classifier shows highest median. In other words, for most of projects, BERT-based classifier perform better than other classifier in terms of F1-score. On the other hand, Transformers-based classifier shows a lower median and the highest inter-quartile-range (IQR). This indicates that the Transformers-based bug classifier learned poorly as compared to

Table 7

MCC of different BC techniques (in %).

Sl No.	Project	Transformers	CodeBERT	DistilBERT	BERT
1	Apache	82.5	88.4	86.2	87.2
2	Eclipse	89.9	93.5	91.0	92.1
3	KDE	86.4	86.6	86.4	87.1
4	LibreOffice	75.9	84.5	83.4	85.7
5	Linux	74.7	80.0	79.3	81.2
6	Mozilla	88.6	87.7	91.6	92.1
7	NetBeans	86.1	82.5	87.2	89.7
8	OpenOffice	77.4	86.3	84.7	80.9

other BC techniques. A possible reason for this poor performance is that the classifier could not learn the contextual representation due to unidirectional attention [16].

It can also be observed from Fig. 4 that CodeBERT achieves the lowest IQR which indicates that CodeBERT shows most consistent performance among all the techniques. Moreover, DistilBERT and BERT also show relatively smaller IQR as compared to Transformers-based classifier. Therefore, it can be deduced from the figure that BERT-based classifiers outperform Transformers-based classifiers as it captures the bidirectional contextual representation of the input sequence. In general, BERT based classifiers achieved the highest performance in terms of the F1-score which indicates BERT classifiers classified more accurately. Therefore, we use this model to compare existing techniques in Section 6.2.

From Tables 5 and 7, it can be observed that the MCC increases over the size of the dataset. This indicates that our model learns better when sufficient data samples are provided. It can also be observed from Tables 6 and 7 that the MCC score is directly proportional to F1-score. This indicates that both F1-score and MCC score can be used interchangeably, as True Negative (TN) occurrences do not affect the MCC score.

We have carried out an analysis using confusion matrix to get more insights into the performance of our classifier. We randomly sampled 800 bug summaries (100 from each class) from the dataset. Since we are dealing with multiclass bug classification, our primary focus was to correctly identify and differentiate among different classes of bugs, rather than correctly identifying what does not belong to any class. Therefore, we focused only on two entities of confusion matrix, that is, True Positive (TP) and False Positive (FP). We have represented these two parameters in Table 8. In Table 8, it can be observed that approximately 86 out of 100 bugs could be correctly classified by our classifier. On the other hand, approximately 14 out of 100 bugs remained misclassified. Moreover, maximum TP is observed in

Table 8

Confusion matrix for our proposed classifier (in count).

Sl No.	Class	True positive	False positive
1	Database	85	13
2	Enhancement	91	17
3	Infrastructure	88	14
4	Logic	87	16
5	Network	86	15
6	Performance	81	10
7	Security	81	12
8	Usability	87	17
–	Average	85.75	14.25

enhancement bugs and the minimum TP is observed for *performance* and *security* bugs. A possible reason for this outcome can be the relative size of the classes. The *enhancement* class has the maximum number of bugs and *security* class has the minimum number of bugs. Similarly, FP is also observed to be maximum for *enhancement* bugs and minimum for *performance* bugs.

Now, we provide a qualitative overview of the performance of our proposed classification technique. Fig. 5 represents the performance of our BERT-based BC technique in terms of AUC-ROC (AR) for the eight considered projects. It can be deduced from Fig. 5 that our technique achieved a mean macro-average ROC of 98.25%. It can also be observed from Fig. 5 that Figs. 5(a)–5(c), 5(f) and 5(g) represent the highest AR (area = 0.99) for *Apache*, *Eclipse*, *KDE*, *Mozilla*, and *NetBeans* projects respectively. However, *LibreOffice*, *Linux*, and *OpenOffice* showed a relatively lower AR. A possible reason for the drop in AR is that the number of bugs in those projects is relatively fewer (please see Table 5).

6.2. Comparison with related work

We compare our work with Logistic Regression (LR) [6], Long Short Term Memory (LSTM) [17], and Attention+LDA [18] techniques. LR is an effective machine learning-based classification technique [6] used for bug classification. Catolino et al. [6] proposed four machine learning-based techniques: Naive Bayes (NB), Support Vector Machine (SVM), Logistic Regression (LR), and Random Forest (RF) to classify 1139 bugs from three open source projects: Mozilla, Eclipse, and Apache. They manually built a taxonomy for these bug reports, resulting in nine classes. Since they aim to classify bugs into several categories, we compared the classification performance of our approach with this LR-based technique.

Although the aim of their work is closely related to ours, there are many notable differences. We performed textual analysis on the bug summaries using a heuristic-based classification technique. We used an earth-mover distance-based technique called Word Movers Distance (WMD) to annotate bug summaries. We also considered eight pre-defined classes selected based on different niche areas in software industries. These classes are selected based on suggestion from eight industry experts. Therefore, we feel that our approach is more useful in practice. We also mined eight open-source software bug repositories through the Bugzilla interface. They are *Apache*, *Eclipse*, *KDE*, *LibreOffice*, *Linux*, *Mozilla*, *NetBeans*, and *OpenOffice*. We collected a massive set containing 1357977 bug summaries. A sample bug from each project and each class is described in Section 4.1. This led to an annotated set of bugs containing the bug summary and its class. The supervised dataset can further be trained for better textual feature learning.

Ye et al. [17] proposed an LSTM-based classification technique to classify bug reports. They evaluated their LSTM-based classifier on 9000 bug summaries over three projects. In their technique, each word in the bug summary is processed sequentially, one at a time. The recurrent unit of LSTM gathers important information when each word is processed. A significant drawback of this technique is vanishing

Table 9

Comparison with existing technique (F1-Score in %).

Sl. No.	Project	LR	LSTM	Attention + LDA	BERT
1	Apache	65.13	78.93	75.37	85.46
2	Eclipse	69.16	85.72	84.12	89.44
3	KDE	66.23	79.59	81.14	85.84
4	LibreOffice	63.23	76.90	77.41	81.50
5	Linux	60.05	67.08	68.15	78.96
6	Mozilla	71.22	83.96	81.06	88.43
7	NetBeans	65.64	81.66	79.04	86.77
8	OpenOffice	58.11	67.97	70.69	81.84

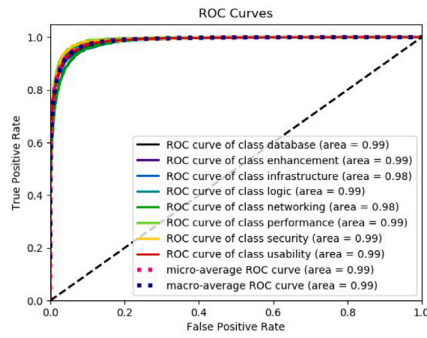
gradient descent, where it loses the importance of words over a long sequence. Unlike processing all the words in the sequence, our approach used an attention-based mechanism [43] that attends to the essential words over the input sequence. Here, the pre-trained Word2Vec [19] embeddings are used to initialize the input bug summaries.

Tang et al. [18] used attention mechanism to classify bug resolution reports into two categories: Bohrbug and Mandelbug. They evaluated the performance of their proposed technique using F1-score and found it to be 73%. Meng et al. [44] have used BERT for bug feature extraction. For bug classification, they have used machine learning techniques (K-Nearest Neighbor, Naive Bayes, Logistic Regression, Support Vector Machine, and Random Forest). It may be noted that, they have experimented using a rather restricted number of data samples (2230). On the other hand, we have used deep learning techniques for classification on a comparatively larger dataset (1357977). Though both the techniques are based on use of BERT tool, but the goals are very different. While Meng et al. [44] have used BERT for feature extraction, our technique uses BERT for bug classification. Therefore, the outcomes of the two techniques are not directly comparable. We have compared our proposed technique with two recently reported work on bug classification: Catolino et al. [6] and Tang et al. [18]. Catolino et al. [6] used Logistic Regression (LR) to classify bugs. On the other hand, Tang et al. [18] used Attention+LDA (Latent Dirichlet Allocation) to classify bugs. To the best of our knowledge, these are two recent works similar to our approach. Therefore, we consider these results for comparison with our experimental classification results.

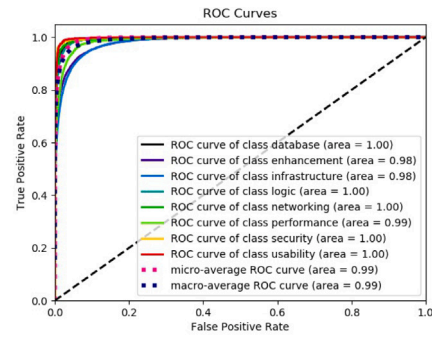
We have experimentally evaluated the performance of the bug classification technique proposed by Catolino et al. [6] (LR), Ye et al. [17] (LSTM), and Tang et al. [18] (Attention + LDA) with our massive curated dataset. For comparison, we consider BERT-based classifier, since it showed the best result among the four proposed attention-based techniques. We compare the performance obtained using LR, LSTM, and Attention+LDA to our proposed BERT-based approach. We have performed t-test hypothesis testing to verify the hypothesis that our technique should exhibit improved learning from the contextual features of the bug resolution summary report due to the use of the attention mechanism. We found $p = 0.034$, 0.025 , and 0.039 ($p < 0.05$) when compared to a reported Machine learning-based (LR) technique [6] and two Deep learning-based (Attention+LDA and LSTM) techniques [17,18], respectively. This ($p < 0.05$) suggests that the null hypothesis is probably false, indicating that the use of deep learning techniques for bug classification should improve classification performance. A possible reason for this performance improvement can be that our technique learns the contextual representation using the attention mechanism.

We afterward used *F1-Score* as the metric for the performance comparison. Table 9 presents the summary of outcome of our experimentation. It can be observed in the table that our technique outperformed the existing techniques, for all projects. However, our technique performed only marginally better for *OpenOffice* project among all the projects.

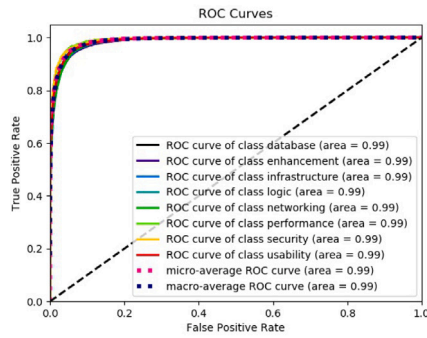
We use Mean Relative Improvement (MRI) to compare the experimental results of our proposed techniques with the related work. First,



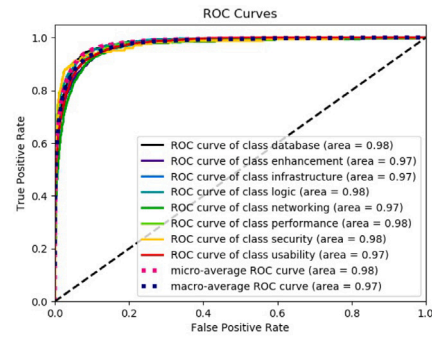
(a) Apache



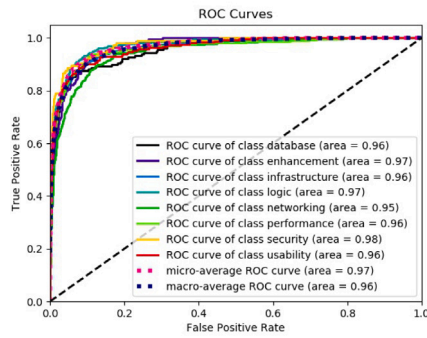
(b) Eclipse



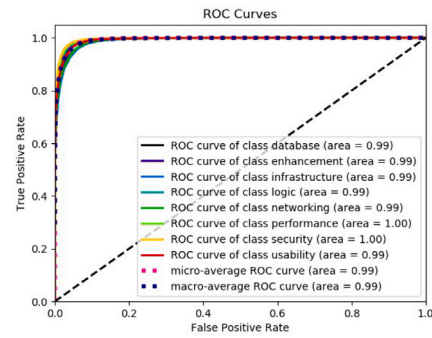
(c) KDE



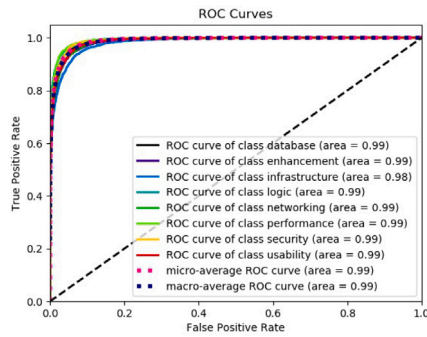
(d) LibreOffice



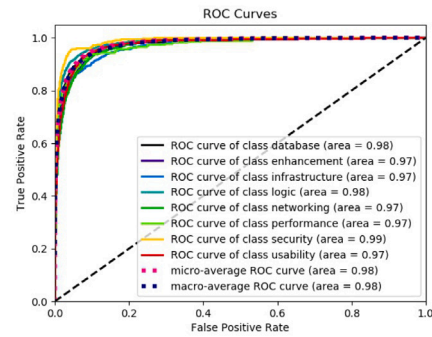
(e) Linux



(f) Mozilla



(g) NetBeans



(h) OpenOffice

Fig. 5. Classifier performance for different projects.

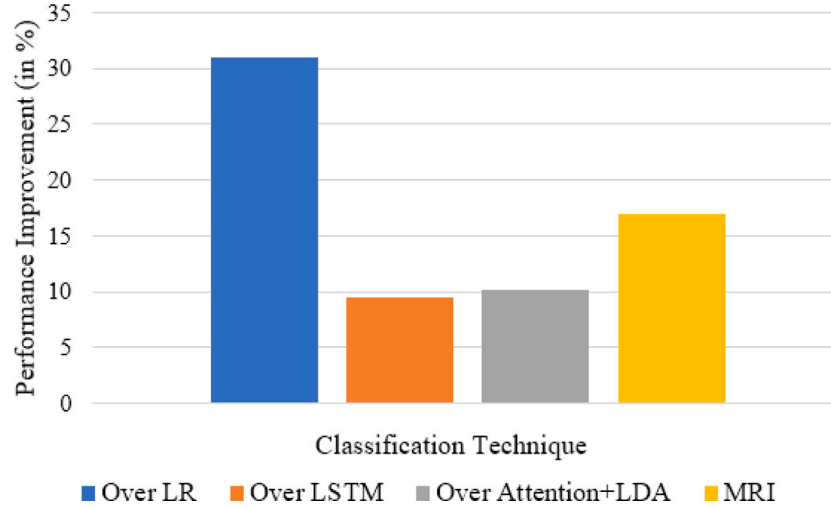


Fig. 6. Overall improvement over existing techniques.

Table 10

Mean Relative Improvement (MRI) over existing technique (in %).

S.N.	Project	Size	LR	LSTM	Attention+LDA	MRI
1	Apache	91 273	31.21	8.27	13.39	17.62
2	Eclipse	320 576	29.32	4.34	6.32	13.33
3	KDE	202 807	29.61	7.85	5.79	14.42
4	LibreOffice	47 511	28.89	5.98	5.28	13.39
5	Linux	14 776	31.49	17.71	15.86	21.69
6	Mozilla	502 989	24.16	5.32	9.09	12.86
7	NetBeans	136 539	32.19	6.26	9.78	16.08
8	OpenOffice	41 506	40.84	20.41	15.77	25.67

we define Relative Improvement (RI) in the following:

$$RI = \frac{\mathcal{V}_{i2} - \mathcal{V}_{i1}}{\mathcal{V}_{i1}} \quad (11)$$

where, $\mathcal{V}_{i1}$ and $\mathcal{V}_{i2}$ are the F1-scores achieved by Technique 1 and Technique 2 respectively. MRI is the average of the relative improvements achieved by Technique 2 over Technique 1.

Table 10 represents the MRI in F1-score over the existing LR and LSTM-based techniques. Here, the *Project* column represents the eight open source projects considered in our study, and the *Size* column shows the respective number of bugs raised in each project. Subsequently, the *LR*, *LSTM*, and *Attention+LDA* columns indicates the relative improvement (RI) in F1-score over LR, LSTM, and Attention+LDA respectively. It can be observed from Table 10 that the RIs are highest over all LR (40.84), LSTM (20.41), and Attention+LDA (15.77) for OpenOffice projects. On the other hand, it is the lowest over LR (24.16) for Mozilla, LSTM (4.34) for Eclipse project, and Attention+LDA (5.28) for LibreOffice project.

It can be inferred from the results that the RIs over LR, LSTM, and Attention+LDA are relatively higher for a smaller dataset. However, the RI over LSTM is lower for relatively larger datasets. The last column of Table 10 represents the mean relative improvement (MRI) over existing techniques. It can be observed that the MRI is maximum for OpenOffice project (with a relatively smaller dataset and minimum for Mozilla project (with a relatively larger dataset). Therefore, it may be concluded that the MRI is inversely related to the size of the dataset.

In Fig. 6, we present the overall improvement in performance (in F1-score) over one machine learning-based classification technique (LR) and two deep learning-based technique (LSTM, and attention+LDA). It can be observed that our proposed technique outperforms LR, LSTM and Attention+LDA techniques by 30.96%, 9.52%, and 10.16% respectively. On an average, our technique outperforms the existing

techniques by 16.88%. It can be inferred from the experimental results presented in Fig. 6 and Table 9 that: (i) the deep learning-based classification technique performs better than a machine learning-based classifier, and (ii) our proposed technique outperforms the existing techniques.

6.3. Discussion and implication

We have carried out our experimentation on a moderately large curated bug classification dataset. For this, we used deep learning-based techniques that can capture the contextual information contained in a bug report. Precision and recall are both important for a multi-class classification task. We have therefore used the F1-score as a suitable evaluation metric. A higher F1-score indicates better precision and accuracy of the classifier. F1-score is robust metric for a balanced dataset. However, it may not always be feasible to have a bug dataset in which the bugs are equally distributed over all classes. Moreover, F1-score is not sensitive to false positive and false negative predictions. Therefore, we also use MCC to measure the performance of our proposed classifier. MCC considers all four metrics such as True Positive, True Negative, False Positive, and False Negative. Consequently, it is considered more suitable for imbalanced dataset. MCC measures the correlation between the actual values and output values, which ranges from -1 to 1 , where 1 indicates a perfect correlation and -1 indicates a counter-correlation. F1-score and MCC can be meaningfully used depending on the distribution of the bugs in the dataset across the considered classes. In other words, F1-score is preferred over MCC where the dataset is balanced and the classifier is not sensitive to false predictions (i.e., False Positive and False Negative). On the other hand, MCC is preferred over F1-score where the dataset is unevenly distributed across the classes and the classifier is sensitive to false predictions.

Our experimental findings demonstrate an average F1-score of 84.78% for the eight projects, which is better than the three currently used methodologies (Fig. 6). This means that, on an average, our method can correctly classify both 85% of positive instances and 85% of negative instances of the bug dataset. It is important to carefully address mis-classifications. We have addressed misclassifications in two ways that project managers and developers in the industry might use.

First, the misclassifications are re-run with the trained model for 7 more epochs with the aim of classifying the misclassified bug reports with the trained model. We selected the Mozilla project since it has the highest number of bug reports among all the eight projects. We are able to correctly classify 36% of the misclassified bug reports. We surmise

that rerunning the trained model can substantially improve the correct classification of bug reports.

Second, we have sufficient reasons to believe that with an increase in the size of the training data, performance would increase [45]. We have performed an ablation study by randomly sampling 10%, 30%, 50%, 70%, and 90% of bug reports from the existing training data. We executed our model for three epochs, keeping other settings intact as discussed in Section 5.3. The F1-score in all five settings was estimated to be 77.36%, 79.45%, 82.09%, 84.89%, and 86.41%, respectively. This result indicates that an increase in the training sample would improve the classification performance. Therefore, the number of misclassifications can be further reduced by training our model with larger data samples.

Our technique can be used to accelerate software life cycle activities such as code inspection, testing, and the bug triage process. Code inspection becomes effective if it is carried out against a realistic checklist of the types of anticipated defects and their expected frequencies. Based on the bug resolution summaries for the previous versions of a project or similar projects, our technique can be used to determine the types of bugs that likely to occur in a project as well as their expected frequencies. This information can make code inspection more effective. Also in the testing process, the types of bugs and their percentages in previous versions of the project or similar projects can be used to guide the testers about the test strategies to use and the composition of the test cases. Our technique can also be helpful in the bug triaging process. For example, our technique can be used on the failure report to determine the type of fault that is causing the failure. This information can be used to assign the bug resolution task to an appropriate team member. Further, identification of the type of bug causing a failure can help the debugging process. For example, if a failure report is identified to be a *security* issue, then possible *security* errors can be investigated to fix the bug.

7. Threats to validity

This section discusses four possible threats to our experimental findings and the approaches we followed to mitigate them.

7.1. Taxonomy validity

The number of classes chosen in our study may be incomplete since the bug can be further categorized into finer classes. However, we limit the classes to eight general bug categories based on different niche areas in the software industry. To ensure the completeness of the taxonomy, we performed an iterative content analysis on 800 randomly sampled bugs. Moreover, we dropped out of class bugs and suggestions. To gain more confidence in the completeness and correctness of taxonomy, we conducted a survey among industrial experts. A few more keywords are added to each bug category as a result of their survey feedback. The final taxonomy with keywords for each class is considered to be understandable and complete.

7.2. Internal validity

We proposed a deep learning-based classification technique. Its performance is directly dependent on the hyperparameters used for the implementation due to its deep computation layers. Therefore, the performance obtained may vary on the implementation environment. To bridge this gap, we tried to keep all the hyperparameters, such as batch size, no of epoch, and learning rate identical for each project to ensure a fair comparison.

7.3. External validity

To make our results more general, we collected bug datasets from eight open-source projects that were of varying sizes and complexities. However, it is possible that proprietary projects can have considerably different distribution of bugs, as they use different development models compared to open source projects. We were unable to experiment using bug datasets for proprietary software, as these are hard to obtain.

7.4. Construct validity

One of the important threats to construct validity is class imbalance. In case of multi-class bug classification, it is natural to have imbalanced classes. For example, enhancement bugs were approximately ten times the security bugs (see Table 5). To ensure the generalizability of our proposed bug classifier with the unbalanced dataset, we have used leave-one-out validation strategy to evaluate the model. Moreover, we have used both qualitative (AUC-ROC) and quantitative (F1-score and MCC) metrics to ensure better performance of our classifier. Moreover, it is possible that some bug reports may be wrongly annotated.

7.5. Conclusion validity

The threats to conclusion validity for our multiclass bug classification depend on multiple factors. We have used a heuristic based approach to build the supervised training data. However, one may argue that the curated dataset is not accurate as it is grounded on a heuristic approach. Although, it uses the computing power of NLP to annotate bugs based on its textual features, manual intervention may be required at times. To address this, we have performed an iterative content analysis of the annotated bugs by experts. We have also used hyperparameter tuning to obtain the optimal combinations of hyperparameters to obtain the desired results.

8. Conclusion

We first proposed a heuristic-based classification technique to annotate a bug dataset containing ~1.36M bug reports of eight open source projects into eight categories. We used the Transfer Learning approach [46] to train four attention-based classification techniques to classify our curated dataset. We experimentally evaluated the performance of our proposed technique and the related techniques in terms of F1-Score, AUC-ROC, and MCC. Our experimental results demonstrate that our proposed classification technique outperformed three related techniques by 16.88% on the average in terms of the F1-score. We therefore feel reasonably confident that our proposed approach can be gainfully used as a preliminary classification tool during bug triage activities. This can help reduce the time and cost overheads associated with software maintenance. In future, we shall target to experiment using different advanced pretrained models and datasets with different characteristics (e.g., size, programming language, proprietary software, etc.)

Declaration of competing interest

We declare that there is not conflict of interest or competing interests.

Data availability

Data will be made available on request.

References

- [1] IEEE Standard Classification for Software Anomalies, in: IEEE Std 1044-2009 (Revision of IEEE Std 1044-1993), 2010, pp. 1–23, <http://dx.doi.org/10.1109/IEEESTD.2010.5399061>.
- [2] R. Mall, Fundamentals of Software Engineering, PHI Learning Pvt. Ltd, 2018.
- [3] S. Mohurle, M. Patil, A brief study of wannacry threat: Ransomware attack 2017, *Int. J. Adv. Res. Comput. Sci.* 8 (5) (2017) 1938–1940.
- [4] K. Sundar, S. Kumar, Blue screen of death observed for Microsoft Windows Server 2012 R2 under DDoS security attack, *J. Inf. Secur.* 7 (04) (2016) 225.
- [5] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schroter, C. Weiss, What makes a good bug report? *IEEE Trans. Softw. Eng.* 36 (5) (2010) 618–643.
- [6] G. Catolino, F. Palomba, A. Zaidman, F. Ferrucci, Not all bugs are the same: Understanding, characterizing, and classifying bug types, *J. Syst. Softw.* 152 (2019) 165–181.
- [7] N. Pinglasai, H. Hata, K.-I. Matsumoto, Classifying bug reports to bugs and other requests using topic modeling, in: *Proc. IEEE 20th Asia-Pacific Soft. Eng. Conf.*, vol. 2, 2013, pp. 13–18.
- [8] P. Terdchanakul, H. Hata, P. Phannachitta, K. Matsumoto, Bug or not? Bug report classification using N-gram IDF, in: *Proc. IEEE Int. Conf. Softw. Maintenance Evol.*, 2017, pp. 534–538.
- [9] M.S. Zolkeply, J. Shao, Classifying software issue reports through association mining, in: *Proc. 34th ACM/SIGAPP Symp. Appl. Comput.*, 2019, pp. 1860–1863.
- [10] I. Chawla, S.K. Singh, Automated labeling of issue reports using semi supervised approach, *J. Comput. Methods Sci. Eng.* 18 (1) (2018) 177–191.
- [11] I. Alazzam, A. Aleroud, Z. Al Latifah, G. Karabatis, Automatic bug triage in software systems using graph neighborhood relations for feature augmentation, *IEEE Trans. Comput. Social Syst.* 7 (5) (2020) 1288–1303.
- [12] Ö. Köksal, C.E. Öztürk, A survey on machine learning-based automated software bug report classification, in: *2022 International Symposium on Multidisciplinary Studies and Innovative Technologies, ISMISIT, IEEE*, 2022, pp. 635–640.
- [13] J.P. Meher, R. Mall, BloomBERT: A deep learning-based cognitive complexity classifier of assessment questions, in: *IEEE International Conference on Advanced Learning Technologies, ICALT, Orem, UT, USA*, 2023, pp. 318–322, <http://dx.doi.org/10.1109/ICALT58122.2023.00099>.
- [14] Y. Arslan, K. Allix, L. Veiber, C. Lothritz, T.F. Bisseyandé, J. Klein, A. Goujon, A comparison of pre-trained language models for multi-class text classification in the financial domain, in: *Companion Proceedings of the Web Conference 2021*, 2021, pp. 260–268.
- [15] M. Kusner, Y. Sun, N. Kolkin, K. Weinberger, From word embeddings to document distances, in: *International Conference on Machine Learning, PMLR*, 2015, pp. 957–966.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] X. Ye, F. Fang, J. Wu, R. Bunescu, C. Liu, Bug report classification using LSTM architecture for more accurate software defect locating, in: *17th IEEE International Conference on Machine Learning and Applications, ICMLA, IEEE*, 2018, pp. 1438–1445.
- [18] Y. Tang, H. Zhou, H. Su, Automatic classification of software bug reports based on LDA and Word2Vec, in: *2022 2nd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology, CEI, IEEE*, 2022, pp. 491–495.
- [19] T. Mikolov, I. Sutskever, K. Chen, G.S. Corrado, J. Dean, Distributed representations of words and phrases and their compositionality, in: *Advances in Neural Information Processing Systems*, vol. 26, 2013.
- [20] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, Y.-G. Guéhéneuc, Is it a bug or an enhancement?: A text-based approach to classify change requests, in: *Proc. Conf. Center Adv. Stud. Collaborative Res.: Meeting Minds*, 2008, pp. 304–318.
- [21] C. Zhou, B. Li, X. Sun, L. Bo, Why and what happened? Aiding bug comprehension with automated category and causal link identification, *Empir. Softw. Eng.* 26 (6) (2021) 1–36.
- [22] N. Pandey, A. Hudait, D.K. Sanyal, A. Sen, Automated classification of issue reports from a software issue tracker, in: *Proc. Prog. Intell. Comput. Techn.: Theory, Pract. Appl.*, 2018, pp. 423–430.
- [23] A.F. Ootom, S. Al jdaeh, M. Hammad, Automated classification of software bug reports, in: *Proc. 9th Int. Conf. Inf. Commun. Manage.*, 2019, pp. 17–21.
- [24] S. Herbold, A. Trautsch, F. Trautsch, On the feasibility of automated prediction of bug and non-bug issues, *Empir. Softw. Eng.* 25 (6) (2020) 5333–5369.
- [25] R. Chillarege, I.S. Bhandari, J.K. Chaar, M.J. Halliday, D.S. Moebus, B.K. Ray, M.Y. Wong, Orthogonal defect classification—A concept for in-process measurements, *IEEE Trans. Softw. Eng.* 18 (11) (1992) 943–956.
- [26] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, C. Zhai, Bug characteristics in open source software, *Empir. Softw. Eng.* 19 (6) (2014) 1665–1705.
- [27] W. Zheng, C. Feng, T. Yu, X. Yang, X. Wu, Towards understanding bugs in an open source cloud management stack: An empirical study of OpenStack software bugs, *J. Syst. Softw.* 151 (2019) 210–223.
- [28] X. Wu, W. Zheng, X. Xia, D. Lo, Data quality matters: A case study on data label correctness for security bug report prediction, *IEEE Trans. Softw. Eng.* 48 (7) (2022) 2541–2556, <http://dx.doi.org/10.1109/TSE.2021.3063727>.
- [29] Y. Fan, X. Xin, D. Lo, A.E. Hassan, Chaff from the wheat: Characterizing and determining valid bug reports, *IEEE Trans. Softw. Eng.* 46 (5) (2020) 495–525.
- [30] J.M. Zhang, M. Harman, L. Ma, Y. Liu, Machine learning testing: Survey, landscapes and horizons, *IEEE Trans. Softw. Eng.* (2020) 1.
- [31] J. Garcia, Y. Feng, J. Shen, S. Almanee, Y. Xia, Q.A. Chen, A comprehensive study of autonomous vehicle bugs, in: *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 385–396.
- [32] S. Robertson, Understanding inverse document frequency: On theoretical arguments for IDF, *J. Doc.* (2004).
- [33] D. Behl, S. Handa, A. Arora, A bug mining tool to identify and analyze security bugs using naive bayes and tf-idf, in: *2014 International Conference on Reliability Optimization and Information Technology, ICROIT, IEEE*, 2014, pp. 294–299.
- [34] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint [arXiv:2002.08155](https://arxiv.org/abs/2002.08155).
- [35] S. Victor, D. Lysandre, C. Julien, W. Thomas, Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter, 2019, arXiv preprint [arXiv:1910.01108](https://arxiv.org/abs/1910.01108).
- [36] J. Devlin, M.W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805).
- [37] K. Weiss, T.M. Khoshgoftaar, D. Wang, A survey of transfer learning, *J. Big data* 3 (1) (2016) 1–40.
- [38] J. Plisson, N. Lavrac, D. Mladenec, A rule based approach to word lemmatization, in: *Proc. IS04*, 2004, pp. 83–86.
- [39] J. Guo, J. Cheng, J. Cleland-Huang, Semantically enhanced software traceability using deep learning techniques, in: *2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE, IEEE*, 2017, pp. 3–14.
- [40] M. Sokolova, N. Japkowicz, S. Szpakowicz, Beyond accuracy, F-score and ROC: A family of discriminant measures for performance evaluation, in: *Australasian Joint Conference on Artificial Intelligence, Springer, Berlin, Heidelberg*, 2006, pp. 1015–1021.
- [41] J. Davis, M. Goadrich, The relationship between precision-recall and ROC curves, in: *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240.
- [42] D. Chicco, G. Jurman, The advantages of the matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, *BMC Genomics* 21 (2020) 1–13.
- [43] T.Y. Yu, C.Y. Huang, N.C. Fang, Use of deep learning model with attention mechanism for software fault prediction, in: *8th International Conference on Dependable Systems and their Applications, DSA, IEEE*, 2021, pp. 161–171.
- [44] F. Meng, X. Wang, J. Wang, P. Wang, Automatic classification of bug reports based on multiple text information and reports' intention, in: *Theoretical Aspects of Software Engineering: 16th International Symposium, TASE 2022, Cluj-Napoca, Romania, July 8–10, 2022, Proceedings, Springer International Publishing, Cham*, 2022, pp. 131–147.
- [45] X. Zhu, C. Vondrick, C.C. Fowlkes, D. Ramanan, Do we need more training data? *Int. J. Comput. Vis.* 119 (1) (2016) 76–92.
- [46] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Xiong, Q. He, A comprehensive survey on transfer learning, *Proc. IEEE* 109 (1) (2020) 43–76.